

On-Demand Home Services Marketplace Platform

Winter 2026 - Distributed Systems Assignment 2

Document Overview

This document outlines the **architecture decisions**, **assumptions made**, and **exact steps required** to run the On-Demand Home Services Marketplace Platform. This system demonstrates microservices architecture, Enterprise JavaBeans (EJB) implementation, asynchronous message processing with RabbitMQ, and REST API integration.

System Architecture

Microservices Overview

The system consists of **5 independent microservices**, each with its own codebase, database, and REST API:

Service	Port	Database	Purpose
User Service	8081	user_db	User management (customers, providers, admins), authentication, wallet management
Service Offer Service	8082	offer_db	Service offer management (create, update, retrieve offers)
Booking Service	8083	booking_db	Booking lifecycle management, saga pattern with wallet deduction and rollback
Notification Service	8084	notification_db	Async notification delivery via RabbitMQ, notification retrieval
Request Matching Service	8085	request_db	Service request creation, matching with available offers (3-person team feature)

Infrastructure

- **RabbitMQ**: Message broker for asynchronous communication (ports 5672, 15672)
- **PostgreSQL**: Database cluster (5 separate instances on ports 5433-5437)
- **Spring Boot 3.2.0**: Framework for all microservices
- **Java 17**: Runtime version

Architectural Decisions

1. Spring Boot + EJB Hybrid Approach

Decision: Use Spring Boot for all microservices with EJB components embedded in the User Service.

Rationale:

- Spring Boot provides excellent REST API support and microservices patterns
- EJB implementation in User Service satisfies assignment requirement (2 EJB types)
- Hybrid approach keeps services decoupled while meeting EJB requirement
- OpenEJB annotations (`@Stateless` , `@Singleton`) work seamlessly with Spring

EJB Implementation in User Service:

- **UserManagementBean** (Stateless): Handles stateless user operations (register, login, add funds, find users)
- **SystemRegistryBean** (Singleton): Manages singleton system state (service categories, registration count, system info)

2. Saga Pattern for Booking Workflow

Decision: Implement booking with compensating transactions for rollback.

Rationale:

- Ensures ACID properties across multiple services
- RabbitMQ enables asynchronous saga orchestration
- If payment fails, wallet deduction is automatically reversed
- Both parties notified of success or failure asynchronously

Booking Flow:

1. Verify offer exists → 2. Check wallet balance
3. Deduct from wallet → 4. Confirm booking
5. Send async notifications → OR → Rollback on any failure

3. Database Isolation Per Service

Decision: Each microservice has its own PostgreSQL database (separate containers).

Rationale:

- Ensures true service independence (no direct database access between services)
- Services must communicate via REST APIs only
- Enables independent scaling and technology choices per service
- Prevents tight coupling through shared databases

4. RabbitMQ for Asynchronous Communication

Decision: Use RabbitMQ message broker for all async operations.

Rationale:

- Decouples services for improved resilience
- Enables reliable notification delivery
- Supports saga pattern rollbacks
- Handles payment failure notifications to admins via direct exchange

Exchange Types Used:

- **Topic Exchange:** For booking confirmations, notifications (routing by service category)
- **Direct Exchange:** For payment failure events (admin notifications)

5. JWT Token-Based Authentication

Decision: Implement JWT tokens for stateless authentication.

Rationale:

- Stateless and distributed-friendly
- No session state needed across services
- 24-hour token expiration for security
- Token includes `role` and `userId` for authorization checks

6. Request Matching as 3-Person Team Feature

Decision: Implement separate Request Matching Service for team feature.

Rationale:

- Assignment requires 3-person teams to implement additional features
- Service isolates matching logic from core booking
- Enables independent scaling of matching operations
- Maintains separation of concerns

Key Assumptions

1. Environment & Runtime

- **Java 17+** is installed and available in `PATH`
- **Maven 3.8+** is installed and available in `PATH`
- **Docker** and **Docker Compose** are installed and running
- Operating system is Linux (or Docker Desktop on Mac/Windows)
- Sufficient disk space (~5GB for Docker images and volumes)

2. Port Availability

- Ports `8081-8085` are available for microservices
- Ports `5433-5437` are available for PostgreSQL databases
- Ports `5672`, `15672` are available for RabbitMQ
- No other services using these ports

3. Authentication & Authorization

- Admin role users are identified by `role="ADMIN"` in JWT token

- Service provider authentication via JWT with `role="SERVICE_PROVIDER"`
- Customer authentication via JWT with `role="CUSTOMER"`
- All admin endpoints require valid JWT token with admin role

4. Wallet & Payment Logic

- Initial wallet balance defaults to `$0.00` if not provided
- Only customers can have wallet balances
- Wallet deduction is immediate on booking creation
- Negative wallet balances are prevented (validation at booking time)

5. Service Request Matching (3-Person Teams)

- Matching logic: `category == category AND price <= offer_price AND date == offer_date`
- Service providers are notified via RabbitMQ of matching requests
- Provider acceptance creates a booking with same saga pattern

6. Notification Delivery

- Notifications are stored in database and retrieved via REST API
- RabbitMQ messages are fire-and-forget (best effort delivery)
- Each notification includes booking details and timestamp
- Notifications support both success and failure scenarios

7. Default Service Categories

System initializes with default categories on startup:

- Plumbing, Carpentry, Electrical, Cleaning, Painting, Landscaping, HVAC, Appliance Repair

Additional categories can be added via admin endpoint.

8. Database Schema

- Hibernate ORM generates schemas on first run (`ddl-auto: update`)
- Primary keys use auto-increment for all entities
- Timestamps automatically managed for created/updated dates
- Foreign keys reference service IDs (UUID format)

Implementation Status vs Requirements

Fully Implemented

Requirement ID	Feature	Status	Evidence
FR-ADM-1	View all registered users	✅	<code>GET /api/admin/users</code>
FR-ADM-3	Add new service category	✅	<code>POST /api/admin/categories</code>
FR-SP-1 to FR-SP-7	Service provider features	✅	Provider registration, offers, completed services, notifications
FR-CUST-1 to FR-CUST-8	Customer features	✅	Registration, wallet, browsing, booking, history
FR-ADD-1 to FR-ADD-2	3-person team features	✅	Service requests, request matching
EJB Requirement	2 EJB types in one service	✅	Stateless + Singleton beans in User Service
Microservices	Independent services with REST APIs	✅	5 microservices with separate databases
RabbitMQ	Async message processing	✅	Booking saga with notifications
REST APIs	All operations via REST endpoints	✅	Postman collection included

Partial Implementation / Design Decisions

Requirement ID	Feature	Note
FR-ADM-2	Transaction history filtering	Endpoint exists (<code>GET /api/admin/bookings</code>), but filtering parameters are optional. Can retrieve all bookings and filter client-side. Query parameters <code>?status=</code> , <code>?startDate=</code> , <code>?endDate=</code> can be implemented if needed.
FR-CUST-8	Booking history filtering	Similar to FR-ADM-2. <code>GET /api/bookings/customer/{customerId}</code> returns all bookings; filtering via query parameters optional.

Step-by-Step Instructions to Run

Prerequisites Checklist

Before starting, verify:

- ✓ Java 17+ installed: `java -version`
- ✓ Maven 3.8+ installed: `mvn -version`
- ✓ Docker installed: `docker --version`
- ✓ Docker Compose installed: `docker-compose --version`
- ✓ Ports 8081-8085, 5433-5437, 5672, 15672 are available
- ✓ At least 5GB free disk space

Option A: Run with Docker Compose (Recommended)

Advantage: Complete isolated environment, no local dependencies needed except Docker.

Step 1: Navigate to Project Root

```
cd /home/mark/last_term/dist_systems/Assi_2/code
```

Step 2: Build Docker Images

```
docker-compose build
```

Expected Output: Docker will build images for all 5 services. **Time:** ~5-10 minutes (first time only)

Step 3: Start All Services

```
docker-compose up -d
```

Expected Output: All containers start in detached mode.

```
Creating user-db ... done
Creating offer-db ... done
Creating booking-db ... done
Creating notification-db ... done
Creating request-db ... done
Creating rabbitmq ... done
Creating user-service ... done
Creating service-offer-service ... done
Creating booking-service ... done
Creating notification-service ... done
Creating request-matching-service ... done
```

Step 4: Verify Services Are Running

```
docker-compose ps
```

Expected Output: All containers should have status `Up` (or `healthy`).

Step 5: Check Service Health

User Service Health Check (waits for Spring Boot to fully start):

```
curl http://localhost:8081/actuator/health 2>/dev/null || echo "Service starting..."
```

Success Response:

```
{"status": "UP"}
```

Test Admin Endpoint (creates first admin user):

```
curl -X POST http://localhost:8081/api/auth/admin/register \
-H "Content-Type: application/json" \
-d '{
  "username": "admin1",
  "password": "admin_password",
  "email": "admin@marketplace.com"
}'
```

Expected Response:

```
{
  "id": 1,
  "username": "admin1",
  "email": "admin@marketplace.com",
  "role": "ADMIN",
  "active": true
}
```

Step 6: View Logs (Optional)

```
# View logs for a specific service
docker-compose logs user-service -f

# View all logs
docker-compose logs -f

# Exit with Ctrl+C
```

Step 7: Stop All Services

```
docker-compose down
```

Note: This stops all services but preserves data volumes. To remove volumes:

```
docker-compose down -v
```

Option B: Run Locally (Linux/Mac)

Advantage: Faster iteration during development. Requires local PostgreSQL and RabbitMQ.

Step 1: Install Local Dependencies

PostgreSQL 15 (Ubuntu/Debian):

```
sudo apt-get update
sudo apt-get install postgresql-15

# Start PostgreSQL
sudo systemctl start postgresql
```

RabbitMQ (Ubuntu/Debian):

```
sudo apt-get install rabbitmq-server

# Start RabbitMQ
sudo systemctl start rabbitmq-server
```

Verify RabbitMQ is running:

```
curl http://localhost:15672/api/overview \
-u guest:guest
```

Step 2: Create Databases

Connect to PostgreSQL and create all 5 databases:

```
psql -U postgres -c "CREATE DATABASE user_db;"
psql -U postgres -c "CREATE DATABASE offer_db;"
psql -U postgres -c "CREATE DATABASE booking_db;"
psql -U postgres -c "CREATE DATABASE notification_db;"
psql -U postgres -c "CREATE DATABASE request_db;"
```

Verify databases created:

```
psql -U postgres -l | grep -E "user_db|offer_db|booking_db|notification_db|request_db"
```

Step 3: Build All Services

Navigate to project root:

```
cd /home/mark/last_term/dist_systems/Assi_2/code
```

Build each service (in parallel terminals or sequentially):

```
# Terminal 1: Build User Service
cd user-service
mvn clean package -DskipTests
cd ..

# Terminal 2: Build Service Offer Service
cd service-offer-service
mvn clean package -DskipTests
cd ..

# Terminal 3: Build Booking Service
cd booking-service
mvn clean package -DskipTests
cd ..

# Terminal 4: Build Notification Service
cd notification-service
mvn clean package -DskipTests
cd ..

# Terminal 5: Build Request Matching Service
cd request-matching-service
mvn clean package -DskipTests
cd ..
```

Alternative (Sequential):

```
for service in user-service service-offer-service booking-service notification-service request-matching-service; do
  cd $service
  mvn clean package -DskipTests
  cd ..
done
```

Step 4: Start Services (Use 5 Terminal Windows)

Terminal 1 - User Service:

```
cd user-service
mvn spring-boot:run
```

Expected Output: Started UserServiceApplication in ... seconds

Terminal 2 - Service Offer Service:

```
cd service-offer-service
mvn spring-boot:run
```

Terminal 3 - Booking Service:

```
cd booking-service
mvn spring-boot:run
```

Terminal 4 - Notification Service:

```
cd notification-service
mvn spring-boot:run
```

Terminal 5 - Request Matching Service:

```
cd request-matching-service
mvn spring-boot:run
```

Step 5: Verify All Services Started

```
# Check each service responds
for port in 8081 8082 8083 8084 8085; do
  curl -s http://localhost:$port/actuator/health | grep -q "UP" && echo "Port $port: ✓" || echo "Port $port: ✗"
done
```

🔍 Testing the System

Using Postman Collections

Two Postman collections are provided:

- HomeServices_Postman_Collection.json - Complete test collection
- home_Services_collection_2.json - Alternative collection

Import into Postman:

1. **Open Postman** → Click "Import" button
2. **Select file** → Choose HomeServices_Postman_Collection.json
3. **Collections tab** → Expand "On-Demand Home Services"
4. **Run requests** → Click individual requests or use Collection Runner

Sample Test Flow:

1. **Admin Registration** → POST /api/auth/admin/register

```
{
  "username": "admin1",
  "password": "admin_password",
  "email": "admin@marketplace.com"
}
```

2. **Admin Login** → `POST /api/auth/login`

```
{
  "username": "admin1",
  "password": "admin_password"
}
```

Save JWT token from response for subsequent requests.

3. **Register Service Provider** → `POST /api/providers/register`

```
{
  "username": "provider1",
  "password": "provider_password",
  "email": "provider@marketplace.com",
  "professionType": "Plumbing"
}
```

4. **Register Customer** → `POST /api/customers/register`

```
{
  "username": "customer1",
  "password": "customer_password",
  "email": "customer@marketplace.com",
  "initialBalance": 500
}
```

5. **Add Service Offer** → `POST /api/offers` (as provider)

```
{
  "category": "Plumbing",
  "description": "Pipe repair and installation",
  "price": 80,
  "availableDate": "2026-05-15"
}
```

6. **Create Booking** → `POST /api/bookings` (as customer)

```
{
  "offerId": "uuid-from-previous-step",
  "customerId": 2
}
```

7. **Verify Notification** → `GET /api/notifications/user/{userId}` **Expected:** Booking confirmation in response.

8. **Check Transaction History** → `GET /api/admin/bookings` (as admin)

📖 API Endpoints Quick Reference

User Service (Port 8081)

Authentication:

```
POST /api/auth/admin/register
POST /api/auth/provider/register (alias for /api/providers/register)
POST /api/auth/login
```

Customers:

```
POST /api/customers/register
GET /api/customers/{userId}/wallet
POST /api/customers/{userId}/wallet/add
```

Providers:

```
POST /api/providers/register
GET /api/providers/{providerId}
```

Admin:

```
GET /api/admin/users
POST /api/admin/categories
GET /api/admin/system-info
GET /api/admin/bookings
```

Service Offer Service (Port 8082)

Offers:

```
POST /api/offers (Create)
GET /api/offers/{offerId} (Get single)
GET /api/offers/category/{category} (Browse by category)
GET /api/offers/provider/{providerId} (Provider's offers)
PUT /api/offers/{offerId}/provider/{providerId} (Update)
GET /api/offers/provider/{providerId}/completed (Completed services)
```

Booking Service (Port 8083)

Bookings:

```
POST /api/bookings (Create booking)
GET /api/bookings/{bookingId} (Get single)
GET /api/bookings/customer/{customerId} (Customer history)
PUT /api/bookings/{bookingId}/status (Update status)
```

Notification Service (Port 8084)

Notifications:

```
GET /api/notifications/user/{userId} (Get user notifications)
GET /api/notifications/{notificationId} (Get single)
```

Request Matching Service (Port 8085)

Service Requests:

```
POST /api/service-requests (Create request)
GET /api/service-requests/{requestId} (Get single)
GET /api/service-requests/user/{userId} (User's requests)
PUT /api/service-requests/{requestId}/match (Accept match)
```

🔧 Troubleshooting

Services Not Starting

Problem: Connection refused or Cannot connect to database

Solution:

```
# Check if services are running
docker-compose ps

# Check service logs
docker-compose logs user-service

# Wait 30 seconds for services to initialize
sleep 30
curl http://localhost:8081/actuator/health
```

Port Already in Use

Problem: Bind exception: Address already in use

Solution:

```
# Find process using port 8081
lsof -i :8081

# Kill process (example: PID 12345)
kill -9 12345

# Or use Docker Compose override
docker-compose down && docker-compose up -d
```

Database Connection Issues

Problem: psql: command not found or role "postgres" does not exist

Solution (local setup):

```
# Verify PostgreSQL running
sudo systemctl status postgresql

# Connect as postgres user
sudo -u postgres psql

# Create superuser (if needed)
sudo -u postgres createuser --superuser postgres
```

RabbitMQ Connection Issues

Problem: Cannot connect to rabbitmq:5672

Solution:

```
# Check RabbitMQ running
sudo systemctl status rabbitmq-server

# Check RabbitMQ logs
sudo tail -f /var/log/rabbitmq/rabbit@*.log

# Restart RabbitMQ
sudo systemctl restart rabbitmq-server

# Verify connectivity
curl http://localhost:15672/api/overview -u guest:guest
```

JWT Token Expired

Problem: 401 Unauthorized - Token expired

Solution: Generate new token by logging in again.

Booking Fails with "Insufficient Balance"

Problem: Booking rejected due to insufficient wallet balance

Solution: Add funds to wallet before booking:

```
POST /api/customers/{customerId}/wallet/add
{
  "amount": 200
}
```

📁 Project Structure

```
code/
├── docker-compose.yml           # Docker Compose configuration
├── HomeServices_Postman_Collection.json  # API test collection
├── home_Services_collection_2.json      # Alternative test collection
├──
├── user-service/                # EJB Implementation (2 EJB types)
│   ├── src/main/java/.../ejb/
│   │   ├── UserManagementBean.java    # @Stateless
│   │   └── SystemRegistryBean.java    # @Singleton
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/resources/application.yml
├──
├── service-offer-service/
│   ├── src/main/java/.../controller/OfferController.java
│   ├── src/main/java/.../service/OfferService.java
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/resources/application.yml
├──
├── booking-service/
│   ├── src/main/java/.../service/BookingService.java # Saga pattern
│   ├── src/main/java/.../client/*.java             # REST clients
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/resources/application.yml
├──
├── notification-service/
│   ├── src/main/java/.../messaging/NotificationListener.java
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/resources/application.yml
├──
└── request-matching-service/
    ├── src/main/java/.../service/RequestMatchingService.java
    ├── pom.xml
    ├── Dockerfile
    └── src/main/resources/application.yml
```

🔒 Security Considerations

JWT Token Security

- Tokens expire after 24 hours
- Secret key: marketplace-secret-key-for-jwt-signing-must-be-long-enough-256bits
- For production, use environment variables for secret keys

Password Security

- Passwords encrypted using bcrypt (PasswordEncoder in Spring Security)

- Never transmitted in plaintext (use HTTPS in production)

Database Credentials

- Default credentials (dev environment): postgres:postgres
- These are visible in docker-compose.yml for testing purposes
- For production, use secrets management (AWS Secrets Manager, Kubernetes Secrets, etc.)

Default Credentials (for Testing)

Role	Username	Password	Purpose
Admin	admin1	admin_password	System administration
Provider	provider1	provider_password	Service provider operations
Customer	customer1	customer_password	Customer booking operations

Known Limitations & Future Improvements

1. **Transaction History Filtering** - Currently returns all bookings. Query parameters for date/status filtering can be added to optimize retrieval.
2. **Notification Retention** - Notifications stored indefinitely in database. Consider implementing TTL/archival policy.
3. **Rate Limiting** - No API rate limiting implemented. Should add per-user rate limits for production.
4. **Audit Logging** - Basic logging via SLF4J. Consider centralized logging (ELK stack) for production.
5. **API Documentation** - No Swagger/OpenAPI documentation. Consider adding springdoc-openapi-ui dependency.
6. **Error Handling** - Global exception handlers implemented but could be enhanced with more specific error codes.
7. **Request Matching Algorithm** - Current matching is exact (category, price, date). Could implement fuzzy matching or price range tolerance.

Support & Troubleshooting Contact Points

For common issues:

- **Database Issues:** Check PostgreSQL logs: /var/log/postgresql/
- **RabbitMQ Issues:** Check logs: /var/log/rabbitmq/
- **Application Logs:** docker-compose logs <service-name>
- **Spring Boot Health:** curl http://localhost:<port>/actuator/health

Final Checklist Before Submission

- ☒ All 5 microservices implemented
- ☒ 2 EJB types in User Service (Stateless + Singleton)
- ☒ RabbitMQ integration for async messaging
- ☒ Independent databases per service
- ☒ REST APIs for all functional requirements
- ☒ Saga pattern with rollback for bookings
- ☒ Postman collections for testing
- ☒ Docker Compose configuration for easy deployment
- ☒ Authentication with JWT tokens
- ☒ 3-person team feature (Request Matching Service)
- ☒ This README with setup instructions

Document Information

- **Created:** May 8, 2026
- **Version:** 1.0
- **Assignment:** Winter 2026 - Distributed Systems (Assignment 2)
- **Team Size:** 3 persons

- **Deployment:** Docker Compose (recommended) or Local Setup
-

End of README